

AI Research Paper Review & Summarization System

Infosys Springboard Internship Project Report

1. Project Overview

The **AI Research Paper Review & Summarization System** is a modular, AI-powered pipeline designed to automate the comparative analysis of multiple research papers. The system retrieves academic papers (automatically or manually), extracts textual content, performs structured comparative analysis using Large Language Models (LLMs), and generates a professionally formatted review report.

The objective of this project is to reduce manual effort in synthesizing academic literature and provide structured, high-quality research summaries using AI-driven workflow orchestration.

2. Problem Statement

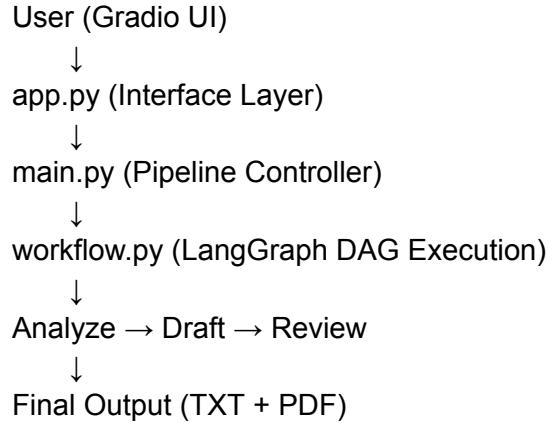
Researchers and students often need to:

- Compare multiple research papers
- Extract methodologies and key findings
- Identify similarities and differences
- Generate structured literature reviews

This process is time-consuming and cognitively intensive. The system automates this pipeline using AI and structured workflow execution.

3. System Architecture

The system follows a **modular pipeline architecture**:



4. Key Technologies Used

4.1 Python

Primary programming language used for backend logic and orchestration.

4.2 Gradio

Used to create an interactive web-based UI allowing users to:

- Enter research topic
- Select input mode (Automatic / Manual)
- Upload PDFs
- Generate, Save, and Download reports

4.3 LangGraph

Used to implement the workflow as a **Directed Acyclic Graph (DAG)**.

Each stage (analysis, drafting, review) is implemented as a node, and execution is handled using:

```
StateGraph  
add_node()  
add_edge()
```

```
compile()
graph.invoke()
```

4.4 Pydantic

Used to define a structured workflow state model:

```
class WorkflowState(BaseModel):
    topic: str
    mode: str
    texts: List[str]
    analysis: Optional[str]
    draft: Optional[str]
    reviewed: Optional[str]
```

This ensures:

- Structured data flow
- Type validation
- Clean separation between workflow stages

4.5 LLM (Gemini)

Used for:

- Comparative analysis
- Draft generation
- Academic refinement

The system ensures the LLM only handles content generation, while metadata formatting is handled at the workflow layer.

4.6 Grandalf

Used for DAG visualization of the research pipeline using the Sugiyama layered layout algorithm.

5. Workflow Design (LangGraph DAG)

The workflow is implemented as a real DAG using LangGraph:

Nodes:

1. Analyze Node
 - Compares research papers
 - Extracts objectives, methods, findings
2. Draft Node
 - Generates structured academic draft
 - Includes: Abstract, Methodology, Results, Insights, Limitations
3. Review Node
 - Refines academic tone
 - Improves clarity
 - Removes redundancy
4. Finalization Step
 - Adds system header
 - Adds timestamp
 - Formats final report

6. Input Modes

6.1 Automatic Mode

- Uses Semantic Scholar API

- Downloads open-access PDFs
- Extracts text automatically

6.2 Manual Mode

- User uploads minimum two PDF files
 - Text extraction performed locally
-

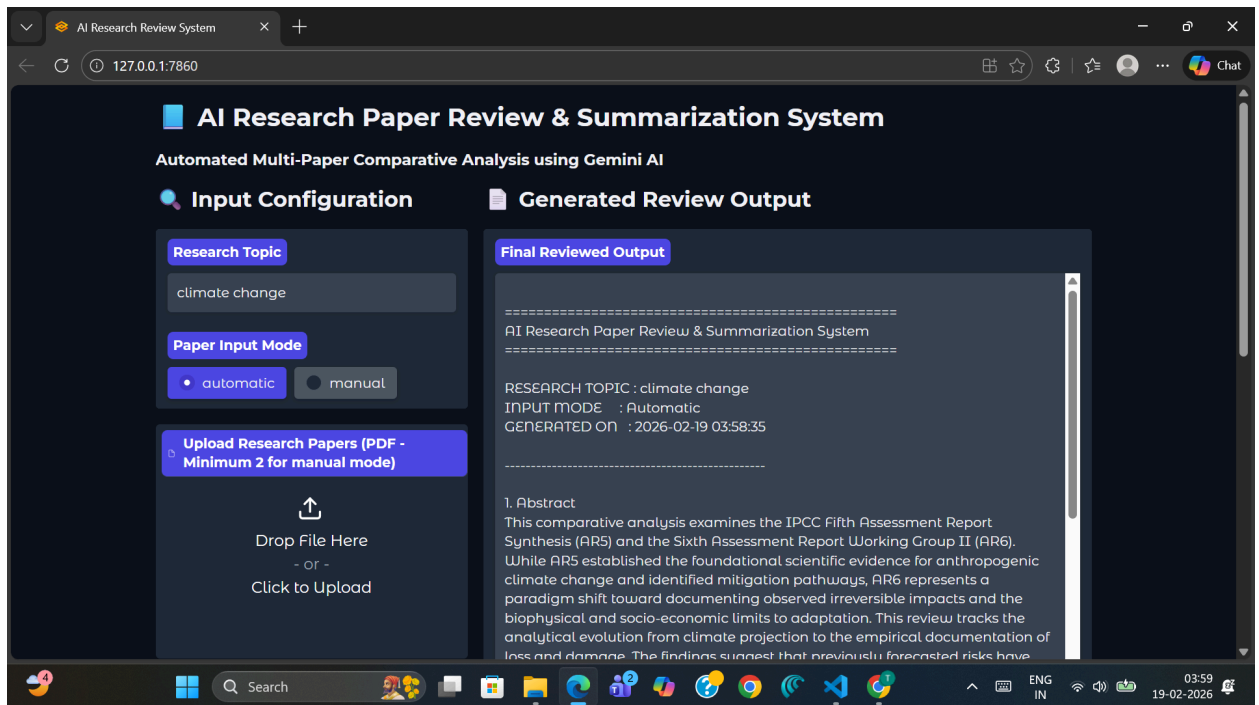
7. Output Generation

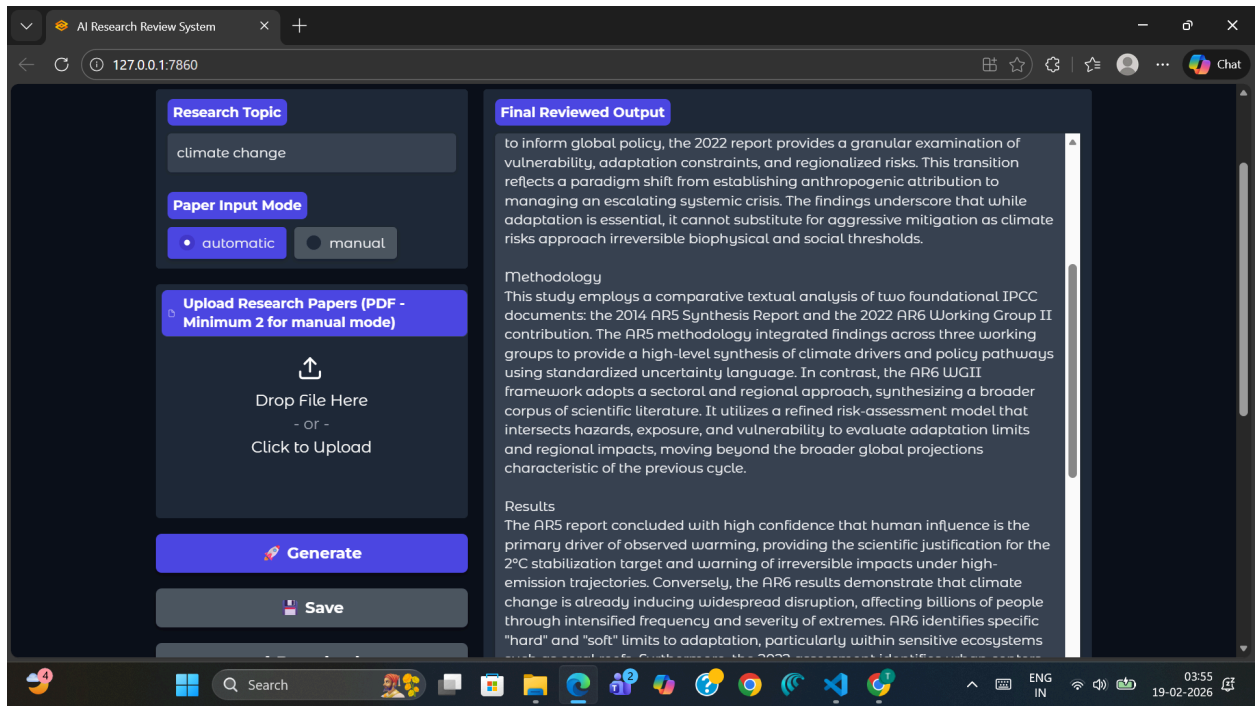
The system generates:

- Structured academic report
- Includes:
 - Research Topic
 - Input Mode
 - Timestamp
 - Abstract
 - Methodology
 - Results
 - Key Insights
 - Limitations
 - References

Users can:

- View output in UI
- Save as .txt
- Save as .pdf
- Download report





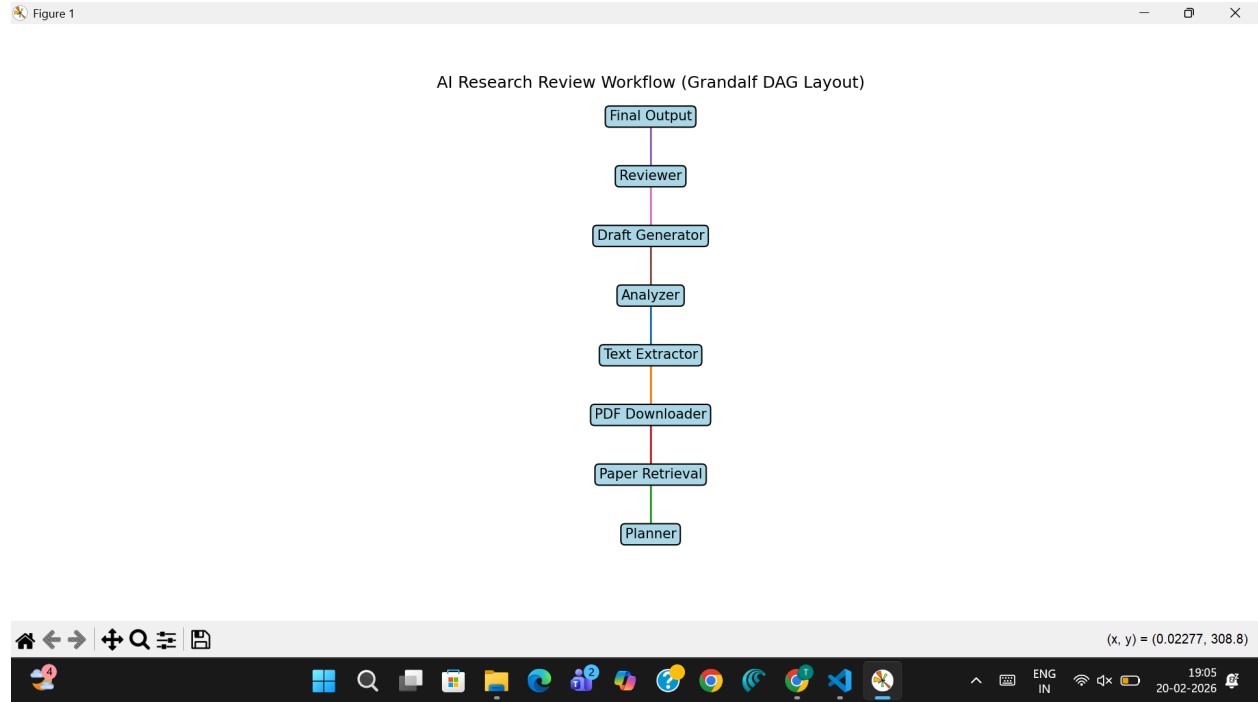
8. Workflow Visualization

The workflow is visualized using Grandalf:

- Directed Acyclic Graph representation
- Layered Sugiyama layout
- Displays processing flow:

Planner → Retrieval → Download → Extract → Analyze → Draft → Review → Output

This visualization helps demonstrate pipeline structure and execution flow.



9. Key Features

- Modular architecture
- Real LangGraph DAG execution
- Pydantic-based state tracking
- AI-powered academic writing
- Structured output formatting
- Manual and automatic input modes
- UI-based save and download functionality
- Workflow visualization

10. Error Handling & Stability

The system includes:

- Try/except blocks in workflow execution
- Validation for minimum paper requirement
- LLM output normalization
- Controlled metadata injection at workflow layer

Header generation is handled after review step to ensure deterministic formatting.

11. Design Principles Followed

- Separation of concerns
 - Modular node-based execution
 - Deterministic metadata control
 - Clean orchestration layer
 - Scalable DAG architecture
-

12. Project Outcome

The system successfully:

- Automates literature comparison
- Generates structured academic reports
- Reduces manual research synthesis effort
- Demonstrates practical implementation of:
 - LLM integration

- LangGraph workflow orchestration
 - Pydantic state management
 - Graph-based architecture visualization
-

13. Future Enhancements

- Streaming LLM responses
 - Deployment on cloud infrastructure
-

14. Conclusion

This project demonstrates a structured and scalable approach to AI-powered research automation. By integrating LangGraph DAG execution, Pydantic state modeling, and LLM-based academic synthesis, the system provides a professional-grade automated literature review tool.

The architecture is modular, extensible, and production-ready, making it suitable for academic and research assistance applications.

Developed by:

Tanuj Kumar
Infosys Springboard Internship Project
AI-based Academic Review Automation System